Business Requirements Document

Product: MedOS HMS v3.0

HIGHLY CONFIDENTIAL

1. Executive Summary

Purpose of the Product: MedOS v3.0 is an ultra-premium, role-based Hospital Management System (HMS) designed to digitize, streamline, and intelligently assist end-to-end hospital workflows ranging from patient registration and clinical documentation to pharmacy dispensing and financial reconciliation.

Core Problem It Solves: Fragmented patient data, revenue leakage through unbilled services, out-of-stock pharmacy incidents, and high cognitive load on doctors. It centralizes disparate health facility operations into a single cohesive, high-performance web platform.

Business Impact: Prevents revenue leakage via auto-billing integration across all clinical modules, reduces prescription errors via AI-assisted advisory, and minimizes operational overhead by enforcing strict role-based access control and inventory FEFO (First-Expired-First-Out) logic.

2. Strategic Context

Market Positioning	Premium, mid-market HMS focusing on modern aesthetics, speed, and AI-augmentation, tailored for high-volume urban clinics and multi-specialty hospitals.
Competitors	Practo Ray, KareXpert, Bajaj Finserv Health, Legacy local desktop HMS softawres.
Differentiation	Ultra-modern "Dark Luxury" glassmorphism UI, integrated LLM-based clinical decision support (Claude AI), strict DPDP (Digital Personal Data Protection) compliance tracking out-of-the-box.

3. Problem Definition

- **Cognitive Overload for Doctors:** Existing HMS UIs are bloated spreadsheets. MedOS solves this via modularized encounter views and AI-based drug suggestions.
- **Inventory Spoilage:** Traditional pharmacy modules lack strict algorithmic enforcement. MedOS strictly enforces FEFO dispensing to minimize expired stock wastage.
- **Disjointed Billing:** Nurses and doctors often forget to log charges for consumables. MedOS auto-posts charges whenever pharmacy dispenses or orders are created.

4. Solution Architecture (High-Level)

System Flow: Client (Browser) ➔ RESTful API ➔ Node.js/Express Backend ➔ SQLite Database.

- **UI Layer (Frontend):** React.js, Vite, Context-based State Management, Custom CSS variables (Dark mode).
- **Auth/Security Layer:** JWT-based Stateless Authentication, BCrypt Hash cryptography, strict API role-guards (`can()` middleware).
- **Business/Data Layer:** Node.js Controllers, Knex.js Query Builder, SQLite single-file persistence.
- **Intelligence Layer:** Anthropic Claude AI integration over external HTTP requests for semantic clinical parsing.

5. Functional Requirements CRITICAL

Module	Core Features	Input / Output Logic
Front Office & Admin	Patient Registration, DPDP Consent, Audit Logging, Role Management	**IN:** Demographics, Consent **OUT:** Generated auto-increment UHID, Patient Profile
OPD & AI Engine	Encounter Notes, Vitals capture, AI Medicine Advisor, Orders	**IN:** Disease narrative **OUT:** Keyword/LLM mapped medicine suggestions from active hospital catalog
IPD (Wards)	Bed Allocation, Visual Occupancy Map, Discharge Summary	**IN:** Patient ID, Room Select **OUT:** Updated Room State, Auto-calculated bed charges upon discharge
Pharmacy Vault	Stock IN (Purchase), Stock OUT (FEFO Dispense), Ledger Drilldown	**IN:** Batch Expiry, Quantity **OUT:** Deducted inventory, auto-generated billing charge item
Financial Billing	Charge generation, Invoice mapping, Unified patient ledger tracking	**IN:** Clinical events **OUT:** GST-compliant Invoices, Accounts Receivable Ledgers

6. UI/UX Logic

Navigation Structure: Persistent Sidebar Left (Role-filtered Context Navigation), Top Header (Contextual Actions & User Profile), Main Canvas (Task-based Workspace).

Interaction Model:

- **Dark Luxury:** Decreases eye strain for continuous 8+ hour usage by healthcare operators (Background `#06060b` gradient).
- **Glassmorphism overlays:** Modals and cards float over context without jarring page reloads.

- **Micro-animations:** Hover lifts, glowing portal effects on login, and toast notifications provide immediate, satisfying feedback ensuring operators feel action completion without reading verbose text.

7. Data & System Design

Storage Assumptions: SQLite is utilized under the assumption of horizontal container constraints where file-based RDBMS suffices for high-read/low-concurrent-write operations. Max DB assumption: ~5GB before archiving.

Processing Logic: Transactions are strongly coupled. Example: `Dispense Pharmacy Item` execution synchronously executes 1) Deduct Stock 2) Increment Patient Outstanding 3) Create Line Item Charge 4) Append Audit Log.

8. User Flow (Step-by-Step Scenario)

> **1. Receptionist** logs in ➔ Registers Patient (acquires DPDP consent) ➔ Books Appointment.

> **2. Nurse** logs in ➔ Views incoming Appointment ➔ Modifies status to "Checked-in" ➔ Captures basic vitals attached to new Encounter.

> **3. Doctor** logs in ➔ Opens Encounter ➔ Uses AI to document chief complaint ➔ Generates Prescription ➔ Digitally signs encounter.

> **4. Pharmacist** logs in ➔ Views queued prescription ➔ Approves dispense (system auto-picks closest expiry batch) ➔ Triggers financial charge.

> **5. Billing** logs in ➔ Aggregates unbilled charges for Patient ➔ Generates Invoice ➔ Process Payment.

9. KPIs / Success Metrics

Business Metrics:	**Product Metrics:**
• Reduction in "Unbilled Revenue" • Decrease in Expired Medicine Write-offs • Reduction in average patient wait time	• DAU/MAU across roles • AI Medicine Suggestion acceptance rate • End-to-end task completion times

10. Non-Functional Requirements

- **Performance:** Initial App mounting < 1.5s. API response boundaries < 200ms (excluding external LLM calls).
- **Scalability:** Designed for single-tenant clinic deployment but architecture allows migration from SQLite to PostgreSQL for multi-tenant scalability.
- **Security:** JWT tokens expiring in 10h, BCrypt hashed passwords, Knex.js parameterized SQL queries (anti-injection), complete Audit trail tracking.

11. Risk Analysis & Scenario Validation

Risk Type	Description	Mitigation
Technical	SQLite Data Locking under hyper-concurrency	Maintain as single-node instance. Migrate schema strictly to PG on scale.
Product	Doctors ignore AI suggestions due to hallucination	System is hard-bounded to ONLY return medicines existing in local `medicine_catalog`.

12. Monetization Model

B2B SaaS / Perpetual License:

1. **Base Instance Fee:** Core modules + 5 Users.
2. **AI Add-on Tier:** Per-query cost pass-through or premium subscription tier enabling Claude API features.
3. **Maintenance SLA:** Annual fee for regulatory module updates (e.g., GST changes, NDHM integration).

13. Decision Triggers (Invalidation Criteria)

This product strategy should pivot or halt if:

- National Health Authority (NHA) mandates cloud-only integration mechanisms incompatible with local instances.
- The local LLM processing API costs outpace the margin generated by premium subscriptions.
- User feedback consistently reports the "Dark Mode" severely limits visibility in bright clinic environments (forcing a light-mode refactor).

14. Hidden Assumptions (CRITICAL)

> **Assumption 1:** The hospital relies on a localized deployment topology where a centralized server runs `localhost` exposed over LAN to client terminals.
>
> **Assumption 2:** Time-series data (like patient wait times) relies on strict client-to-server time synchronization.
>
> **Assumption 3:** The pharmacy operates purely digitally; physical stock variance (theft/loss) is manually corrected as there is no IoT/Barcode integration in v3.0.
>
> **Assumption 4:** The API connection to Anthropic is reliable and patient identifiers are sanitized locally before being pushed to external LLMs.